

When Does a Kernel Win Surface? A Cross-Backend Catalog of Proven Levers, Measured Dead-Ends, and a Bottleneck Taxonomy for Quantized LLM Decode and Prefill

Hanzo AI — hanzo-engine / hanzo-ml technical white paper, 06

Consolidation pass: kernel work measured 2026-06

Abstract

Companion paper 05 established the *model-level* result: hanzo-engine reaches `llama.cpp` decode parity ($0.98\text{--}1.04\times$) and a bounded prefill gap ($0.76\text{--}0.91\times$) on AMD, NVIDIA, and Apple from one multi-backend build. This paper documents the *kernel-level* engineering underneath that result — the optimizations we wrote, measured, and *kept* in hanzo-ml/hanzo-engine, and, just as importantly, the ones we wrote, measured, and *reverted*. We organize roughly thirty controlled experiments (ROCm/gfx1151, CUDA/GB10, Vulkan/gfx1151, Metal/M4 Max) into a four-regime **bottleneck taxonomy** — bandwidth-roofline decode, compute-bound prefill, host/launch-bound decode, and GPU-busy-bound decode — and show that the taxonomy *predicts* whether a given kernel win surfaces end-to-end. The strongest evidence for the taxonomy is that the *same* optimization is a real lever in one (platform, regime) and a measured non-lever in another: int8 **dp4a** lifts ROCm i-quant decode $2.27\times$ (compute-bound) but is end-to-end flat on GB10 (host-bound); a fused router gives ROCm $+13\%$ but CUDA 0% (no expensive sort to eliminate); a fused-coopmat tensor-core prefill is correct but $3.8\times$ slower than int8-ALU **dp4a** on gfx1151 (decode-bound). The governing rule, stated once and tested repeatedly: *a fusion or kernel change helps end-to-end only if it eliminates work in the binding resource; cutting launches or FLOPs in a non-binding resource is flat at best and negative if it injects redundant work*. Every win we report is gated bit-exact against a CPU reference; every A/B is device-map-controlled or kernel-isolated. We close with the one frontier this program has fully characterized but not yet closed — Vulkan *decode*, which we show by profiling is op-count-serialization-bound (1336 dispatches / 903 global barriers / 143 ms GPU fence-wait per token, with CPU recording effectively free) — and the fusion roadmap that the ROCm path already validates.

Contents

1	Introduction: one engine, every backend, and a measurement question	2
2	Architecture: three orthogonal axes and a bit-exact invariant	3
3	A bottleneck taxonomy for single-stream quantized inference	3
4	Kept levers: ROCm (gfx1151, Strix Halo)	4
4.1	Unified dp4a decode cores (the K-quant and i-quant families)	4

4.2	Capture-clean strided ops $\rightarrow$ HIP-graph decode	5
4.3	Indexed-MoE decode without a host round-trip	5
4.4	The MoE graph bug: a position-refresh root-cause, not a buffer bug	5
4.5	Fused indexed-MoE prefill GEMM: the $2.58\times$ prefill lever	5
4.6	Fused MoE routing: <code>moe_route</code> (+13%)	6
4.7	Decode op-fusion that eliminates work (+2.8% each, stacked)	6
5	Kept levers: CUDA (GB10, Grace-Blackwell sm_121)	7
5.1	i-quant completeness: IQ1.S MMQ prefill, TQ1/TQ2 <code>dp4a</code> decode	7
5.2	<code>unpack_ksigns</code> : the real GB10 i-quant kernel wall ($1.71\times$)	7
6	Kept levers: Vulkan (gfx1151) — the prefill GEMM arc	7
6.1	The deadlock: a missing uniform GEMM, surfaced as a liveness bug	7
6.2	The L1 GEMM ladder: 1D dead-end $\rightarrow$ 2D-f32 $2.06\times \rightarrow$ <code>dp4a</code> -2D $9.35\times$	8
7	Measured dead-ends: the boundary map	9
8	The cross-backend law and model-level dilution	9
9	A correctness boundary: the flash-attention repetition bifurcation	10
10	The characterized-but-open frontier: Vulkan decode	11
10.1	Roadmap: the campaign ROCm already validated	11
11	Verification methodology (so the numbers mean something)	12
12	What this paper does <i>not</i> claim	12
13	Conclusion	12
A	Reproducibility	13

1 Introduction: one engine, every backend, and a measurement question

hanzo-engine runs CUDA, ROCm, Metal, Vulkan, and CPU behind one model loader; hanzo-ml is the tensor/kernel layer beneath it. The portability thesis is only credible if the *same* code is competitive with the hand-tuned, single-vendor baseline (`llama.cpp`) on each backend. Paper 05 showed it is, at the model level. The engineering question this paper answers is the one that actually consumes the work: **given a kernel that is provably faster in isolation, will it make the end-to-end token rate go up?**

The naive assumption — faster kernel $\Rightarrow$ faster model — is wrong often enough to be the central hazard of inference optimization. A $9\times$ matmul can be flat end-to-end; a $5\times$ i-quant kernel can fully surface on one GPU and vanish on another; cutting half the launches can lose 22%. The difference is always *which resource is binding*. We therefore present the work not as a list of tricks but as a taxonomy that tells you, before you write the kernel, whether the win can possibly surface — and we validate it on the cases where it predicted “no” and we measured “no.”

What this paper contributes.

1. A **bottleneck taxonomy** (§3) for single-stream quantized inference, with a falsifiable surfacing rule.
2. A **catalog of kept levers** (§4–6) per backend, each with its bit-exact gate and a device-map-controlled or kernel-isolated A/B.
3. A **catalog of measured dead-ends** (§7) — the negatives are the contribution: each one is explained by, and so confirms, the taxonomy.
4. The **Vulkan-decode frontier** (§10): a profile-backed diagnosis and the fusion roadmap that ROCm has already walked.

2 Architecture: three orthogonal axes and a bit-exact invariant

The kernel surface is a product of three axes that we keep deliberately orthogonal, because collapsing them is what produces an unmaintainable $N \times M \times K$ explosion of hand-written kernels:

- **Quant format** (the *value*): the GGUF block layout — how 1–8 bits/weight plus per-block scales reconstruct a float. 22 families are wired (Q8_0/Q4_0/Q4_K/Q5_K/Q6_K/Q2_K/Q3_K, the IQ1/IQ2/IQ3 codebook quants, the TQ ternary quants, etc.).
- **Contraction** (the *operation*): mat-vec (decode, $M=1$), GEMM (prefill, $M>1$), and their indexed-MoE twins (gather/scatter over routed experts).
- **Backend** (the *functor*): how a contraction lowers to ROCm/CUDA/Metal/Vulkan instructions and memory.

On ROCm this is already decomplexed to one generic contraction with the type injected as a trait: `qmatvec_core<WTYPE,XT>` (decode) and `qmmq_core<WTYPE,MOE,NWAVE_M>` (prefill), each instantiated per format, with the MoE variant differing only by an `if constexpr (MOE)` gather/scatter so the dense path codegens byte-identically. The same 22 formats decode through one core; the int8-dp4a fast path is a second core selected by a single trait predicate `RocmQuantType::dp4a_active()`. The forward graph never branches on “is this Q4_K” — dispatch is by kernel-name string, so adding a format is additive.

The invariant that makes aggressive kernels safe. Every kernel — every dp4a reinterpretation, every fusion, every tiling — is gated against a CPU `to_float` oracle to `nbad=0` (bit-exact, modulo documented f32 reorder / f16-rounding tolerances). This is not a test suite bolted on after; it is the precondition for shipping a kernel at all. The reason the catalog below can be this aggressive (reinterpreting a 6-bit asymmetric quant as signed int8, folding a residual-add into an RMS reduction, quantizing activations to int8 mid-GEMM) is that each transform is proven equivalent before it is allowed to matter for performance. The oracles are named inline (`qmatvec_unified_numeric`, `moe_qmmq_numeric`, `add_rmsnorm_numeric`, `qk_rms_norm_rope_numeric`, `moe_route_numeric`, ...) and are the reusable gate for every follow-up.

3 A bottleneck taxonomy for single-stream quantized inference

Single-stream decode and prefill bind on different resources, and within decode the binding resource differs by platform and by whether HIP/CUDA graphs are active. We name four regimes.

Table 1: The four regimes. “Surfacing condition” is what a kernel change must do to move the end-to-end token rate.

Regime	Binding resource	Surfacing condition
Bandwidth-roofline decode	weight bytes / memory BW	Only fewer weight bytes moved (a smaller quant). Kernel arithmetic changes are flat once at roofline.
Compute-bound prefill	matrix/tensor-core FLOPs	Higher effective FLOP/s: better tiling, tensor cores, higher M-tile fill.
Host/launch-bound decode (eager, or graph-less)	per-token CPU dispatch / kernel launch	Fewer launches, or graph capture that hides launch latency. GPU-time cuts are flat.
GPU-busy-bound decode (graphs on)	sum of small-kernel GPU time	Eliminating GPU work (a true fusion). Repackaging the same work is flat.

The surfacing rule (falsifiable).

A kernel change improves the end-to-end token rate only if it reduces consumption of the binding resource. Reducing a non-binding resource is flat; reducing a non-binding resource while adding work to the binding one is negative.

Three corollaries drive the rest of the paper. (i) On a bandwidth-roofline decode, the matvec is already at the memory ceiling, so faster arithmetic is flat — the only decode lever is reducing *kernel count* (in the launch- or busy-bound sub-regimes), not making any one kernel faster. (ii) “Fewer launches” helps only in the launch/busy-bound regimes, and only if the launch you removed was not already free (under graphs it is). (iii) The same kernel can flip regime across platforms: a GPU with more launch overhead relative to memory bandwidth is launch-bound where another is bandwidth-bound, so a launch-cut surfaces on one and not the other.

4 Kept levers: ROCm (gfx1151, Strix Halo)

ROCm is the most mature backend and the one where the taxonomy was first forced on us. All decode A/Bs use the device-map pin `-n 0:48` (the auto-mapper otherwise mis-reads unified memory and offloads $\sim 1/3$ of layers to CPU, which silently corrupts the comparison — a measurement hazard worth more than any single lever).

4.1 Unified dp4a decode cores (the K-quant and i-quant families)

The decode matvec for K-quant reinterprets the block as int8 and uses the RDNA3.5 dot-product builtin (`__builtin_amdgcn_sdot4`; the plain `sdot4` does not compile on gfx1151, which lacks the `dot1-insts` feature). The win is per-format and depends entirely on the regime:

- **Q6_K dp4a**: a $4.9\times$ *kernel* win ($260 \rightarrow 53 \mu\text{s}$ per call) by reading the 16-element scale group as four packed int reads and reconstructing 4 weights/int, one branch per group not per element. The q6 centering ($q - 32$) is handled as a bias term $\sum (q - 32)u = \text{dp4a}(q, u) - 32 \text{dp4a}(1, u)$ because gfx1151 has no per-byte `__vsubss4`. Total GPU kernel time -48% . **Model-level:** $+17\%$ ($28.2 \rightarrow 33.0$ T/s) — it surfaces because eager MoE decode here is partly compute-bound on the slow Q6_K ($\text{output.weight} + \text{attn.v} + \text{ffn.down experts} = 37.7\%$ of GPU work).

- **IQ codebook dp4a (IQ2_XXS/XS/S, IQ3_XXS/S, IQ1_S/M):** $2.27\times$ *end-to-end* ($30.4 \rightarrow 67.7$ T/s on Qwen3-30B-A3B-IQ3_XXS), *beating* llama.cpp-HIP (67.6). The grid coords land in int8 and **dp4a** against the once-quantized activation; signs without `_vsub4` use $\sum (-1)^s g u = \text{dp4a}(g, u) - 2 \text{dp4a}(g \& \text{mask}(s), u)$. Unlike Q6_K, this win fully surfaces *and* is graph-flat — i-quant decode was *compute-bound* (per-element f32 dequant + MAC), so the kernel *was* the wall. This is the textbook “compute-bound decode” case of the taxonomy.

The contrast between Q6_K (kernel $4.9\times$, model $+17\%$) and IQ3 (kernel $2.27\times$, model $2.27\times$) is the taxonomy in one comparison: the i-quant decode was compute-bound so the kernel win *is* the model win; the K-quant decode is mostly bandwidth-bound so only its compute-bound fraction surfaces.

4.2 Capture-clean strided ops $\rightarrow$ HIP-graph decode

The single highest-leverage ROCm change was not a matmul. Decode under HIP graphs hides per-launch host latency, but capture kept failing on the first strided op: each `Map/Reduce/cast/copy` uploaded its tiny dims/strides metadata array host $\rightarrow$ device every op every token, which is an implicit capture violation (HIP 906). The fix rides the metadata *inline by value* in the kernel param block (`#[repr(C)] DimsStrides { v: [usize; 32] }, 256 B`), so zero `clone_htod` fires on the strided path. This both enabled graph capture *and*, by removing a blocking H2D from the eager hot path, lifted eager decode from ~ 11 to $\sim 16\text{--}30$ T/s on its own. It is the enabling lever beneath every launch-bound decode win.

4.3 Indexed-MoE decode without a host round-trip

MoE decode batches all routed slots into one launch over `grid.y`; the expert id is read *on device* (the kernel takes `ids: &RocmStorage`, not `&[u32]`), so no `to_vec1/copy_from_host` round-trip fires per token. This eliminated the per-expert host launch loop for mixed-precision GGUF (Qwen3-30B-A3B has Q6_K `ffn_down` experts inside a Q4_K model), $10.22 \rightarrow 12.07$ T/s.

4.4 The MoE graph bug: a position-refresh root-cause, not a buffer bug

MoE decode under graphs produced garbage from $\sim$ token 12 (“Let. Let. 1 1 1”). The prior hypothesis (recycled routing buffer) was disproven by an env-gated replay-vs-eager `max_abs_diff` probe: dense paths diffed 0.0, MoE diffed 4–16 from token 0, and bisecting the MoE forward (even forcing the experts to identity) still diverged — proving the divergence was in the *shared attention path*. The MoE model baked the decode RoPE position from the host offset at capture and never refreshed it, so every replayed token rotated at the frozen warmup position. The dense path already read RoPE off a stable device tensor (`metadata.rope_positions`) refreshed in place each token; the MoE model simply never got that fix. Threading `positions: &Tensor` through `forward_attn` made MoE graphs replay bit-identically (340-token coherent, $\times 3$). The lesson is methodological: a replay-vs-eager numeric probe localizes a graph bug faster than any amount of staring at buffers.

4.5 Fused indexed-MoE prefill GEMM: the $2.58\times$ prefill lever

MoE prefill was the *entire* deficit versus llama.cpp-HIP ($0.34\times$ while dense was already $1.12\times$). Root cause (`rocprofv3`): routed prefill went through the per-slot decode matvec, so every routed token re-read its expert’s whole weight and ran *no* matrix cores — 70.7% of GPU time on MoE

matvecs. The fix is an expert-grouped WMMA GEMM (`qmmq_core<WTYPE,true>`, the same 16-warp/128×128 int8-WMMA prefill machine with `if constexpr` (MOE) gather/scatter): tokens are counting-sorted by expert host-side (prefill is never captured, so the `ids DtoH` is free) into ≤ 128 -row tiles, and each block stages *one* expert’s weight and amortizes it over all that expert’s tokens (llama’s `mul_mat_id`). **Result:** 366 $\rightarrow$ 944.7 T/s (2.58×), 0.89× of llama; GPU MoE-matmul time -78% . The A/B (`HANZO_MOE_QMMQ_FALLBACK=1` $\Rightarrow$ 365.8) confirms the kernel is the *sole* lever.

Dynamic M-tile (the bandwidth/compute balance point). At ~ 64 tokens/expert a 128-row WMMA tile is half-empty. Parametrizing the core on the M-wave count (`qmmq_core<WTYPE,MOE,NWAVE_M>`, `TILE_M` 128/64/32 with thread-count-agnostic staging loops so the dense path stays byte-identical) and picking `TILE_M` by routed fill lifts M-fill 49.9 $\rightarrow$ 68.2% but throughput only $+2.4\%$ — because the full-prefill MoE GEMM is *weight-bandwidth-bound*, not WMMA-compute-bound, *proven* by the A/B: `TILE_M=32` (even less empty WMMA work) is *slower* because it re-reads each expert weight in two tiles. `TILE_M=64` (one tile per ~ 64 -token expert, minimal weight re-reads) is the balance point. The residual gap to llama is irreducible expert-weight fetch, not tile fill. This is a clean instance of the surfacing rule: more compute headroom is flat when bandwidth binds.

4.6 Fused MoE routing: `moe_route` (+13%)

One kernel (one block/token: shared-mem softmax $\rightarrow$ top- k $\rightarrow$ normalize) replaces the softmax $\rightarrow$ sort $\rightarrow$ narrow $\rightarrow$ sum chain (~ 6 launches/layer $\times 48 \approx 240$ fewer kernels/token). 52.9 $\rightarrow$ 59.9 T/s (+13%), 0.90× llama; long-context run-loop $+15\%$. This is the launch/busy-bound decode lever the bandwidth-wall analysis pointed to, and it surfaces on ROCm precisely because the `amdgcg gpu-sort` it eliminates is disproportionately expensive (a fact that, foreshadowing §7, makes the *same* fusion flat on CUDA).

4.7 Decode op-fusion that eliminates work (+2.8% each, stacked)

`rocrprofv3` of the shipped decode showed it is **GPU-busy-bound** under graphs (12.17 ms busy = 94.5% of the 12.87 ms wall), with non-matvec busy time spread across `quantize_q8_1` (378 μ s), `rmsnorm` (330), `route` (297), `casts` (377), etc. Two fusions that *eliminate* work (not just launches) each surfaced:

- **gate+up shared-quantize** (`moe_matvec_pair`): gate and up consume the same routed token, so broadcast + quantize it *once* and `dp4a` both banks. -48 quantize, -48 ucopy per token.
- **add rmsnorm**: $s = x + \text{residual}$ then `rmsnorm(s) ·  $\alpha$` in one reduce launch, vs. a separate `badd` then `rmsnorm`.
- **fused q/k RMSNorm + positions-RoPE** (`rope_norm_positions`, one block per head-vector): one kernel replaces two standalone norms + RoPE *and* the q/k intermediate write+read. Same f32 math, no redundant work. $+2.8\%$ (77.4 $\rightarrow$ 79.5 T/s), the true-elimination twin of `moe_route`.

Stacked, 1702 $\rightarrow$ 1558 kernels/token; decode past llama-HIP. Each is bit-exact. The discipline: fuse only when the fusion *removes* GPU work — which is exactly what the next section’s biggest dead-end violated.

5 Kept levers: CUDA (GB10, Grace-Blackwell sm_121)

5.1 i-quant completeness: IQ1_S MMQ prefill, TQ1/TQ2 dp4a decode

Two correctness/coverage gaps closed, both bit-exact (`cuda_iquant_tests` 13/13). IQ1_S had native `dp4a` decode but was MMQ-excluded at prefill only because its GPU grid table was a zeroed stub mis-sized at [512] (the indexer needs 11 bits = 2048); filling the real `iq1s_grid_gpu[2048]` verbatim from `llama.cpp` enabled the whole int8-WMMMA prefill machine that was already ported. TQ1.0/TQ2.0 ternary got native `dp4a` decode — symmetric $\{-1, 0, 1\}$ packs straight to signed int8 with *no* bias-sum term; notably `llama.cpp` has no CUDA ternary path at all, so this is pioneering and the ROCm scalar `qdec` is the layout oracle.

5.2 unpack_ksigns: the real GB10 i-quant kernel wall (1.71×)

The i-quant decode matvec was $\sim 75\%$ of GB10 i-quant decode GPU-time. Diffing against `llama.cpp`'s `vec_dot_iq2_xxs_q8_1` found a ROCm-port artifact: the sign codebook was a *divergent __constant__* gather (each lane a different index, serialized through the constant cache). `llama` computes it table-free — the 7-bit index's 8th sign is the popcount-parity of the other 7:

```
// bit-identical to KSIGNS_IQ2XS_D[v], no constant-cache gather:  
unpack_ksigns(v) = (v ^ ((__popc(v & 127) & 1) << 7)) * 0x01010101;
```

Kernel A/B (4096², GB10): IQ2_XXS 84.67 $\rightarrow$ 49.49 μ s = 1.71 $\times$ (51 $\rightarrow$ 87 GB/s); IQ3_XXS 1.40 $\times$; IQ2_XS 1.25 $\times$. The control types that have no gather to kill (IQ2_S/IQ3_S, raw-byte signs) stay *flat* — which *isolates* the gather as the cause and corrects an earlier “47 GB/s VALU wall” conclusion that had measured the baseline and mislabeled it.

The honest model-level result: a GB10 non-lever. Same-engine A/B on Qwen3-4B-UD-IQ2_XXS: **flat** both graphs-on (66.1 vs 66.2 T/s) and eager (60.2 vs 60.2). The 1.71 $\times$ kernel win does not surface because GB10 hanzo i-quant decode is **host/launch-overhead-bound, not matvec-bound** — the GPU finishes the faster matvec and stalls on per-token host work. This is the same kernel (`dp4a`/codebook decode) that fully surfaced on ROCm-eager (§4); the regime flip across platforms is the taxonomy's sharpest prediction. We ship it anyway: bit-exact, it drops a 128-entry table and a divergent gather, it matches `llama`, and it is a latent win for genuinely matvec-bound regimes. The actual GB10 decode lever is launch reduction (the dense decode-graph gate, +6.8%).

6 Kept levers: Vulkan (gfx1151) — the prefill GEMM arc

Vulkan is the youngest backend and the one where we walked the full optimization arc on a single kernel (Q4_K prefill GEMM) in the open, keeping every rung — including the negative ones — as a documented ladder. This is the most complete single-kernel case study in the program.

6.1 The deadlock: a missing uniform GEMM, surfaced as a liveness bug

hanzo-Vulkan *hung* at the first long prefill of any GGUF on gfx1151, while `llama.cpp`-Vulkan and hanzo's own ROCm ran fine. `gdb` of the live hung process put the engine thread in the kernel at `DRM_IOCTL_AMDGPU_GEM_CREATE  $\leftarrow$  QTensor::dequantize  $\leftarrow$  prefill`. The prefill `else` branch dequantized each weight to a fresh ~ 100 –235 MB f32 buffer; under the deferred single command batch *none* free until the end-of-forward flush, so a dense prefill re-expands the whole model to f32

(~32 GB for an 8B) in fresh allocations — on a unified-memory carveout this exhausts memory and `GEMM_CREATE` blocks on a free that can only happen *after* the still-recording batch flushes: a self-deadlock. ROCm never hit it because ROCm prefill rides the int8-WMMA GEMM and never dequantizes — exactly the asymmetry that pointed at the fix. `vulkan_prefill_gemm_max_rows` now returns `usize::MAX` for every dtype that *has* a native quantized GEMM, so prefill always uses the GEMM and never the f32-dequant fallback. The gate is now effectively a “has a GEMM kernel?” predicate; the real cleanup (§10.1) is to give all 22 formats a GEMM so the fallback — and the bug — become impossible by construction.

6.2 The L1 GEMM ladder: 1D dead-end $\rightarrow$ 2D-f32 $2.06\times \rightarrow$ dp4a-2D $9.35\times$

We measured each rung kernel-isolated (`bench_matmul_q4k`: upload operands once, loop the GPU GEMM, one final synchronize — the host-wrapper bench’s per-iter 8 MB upload+readback otherwise masks the kernel). Numbers are $m=512$, $n_{\text{out}}=k=4096$ (17.2 GFLOP).

Table 2: Vulkan Q4_K prefill GEMM ladder, kernel-isolated (gfx1151). Default = the legacy column-per-invocation kernel.

Kernel	Time (ms)	vs. default	Verdict
Legacy column-per-invocation	38.5	$1.0\times$	baseline
1D weight-only LDS stage	223.7	$0.21\times$	dead-end (§7)
2D-tiled f32 (64×64 , BK=32)	19.0	$2.06\times$	kept (universal fallback)
2D-tiled int8 dp4a	3.4	$9.35\times$	shipped default
Naive coopmat (tensor cores)	17.4–19	$0.26\times$ vs dp4a	dead-end (§7)

The shipped kernel (`mul_mm_q4k_tiled_dp4a.comp`) stages a 64×64 output tile: per K-step (BK=32, one Q4_K sub-block) it stages the n -cols’ 32 q4 codes + per-block d_1/m_1 and the m -rows’ int8 activation $xq + xs/xsum$ into LDS, and each of 256 threads **dp4a**s its 4×4 sub-tile via the $Q4_K\times q8_1$ affine identity

$$\text{acc} += d_1[\text{col}] \cdot xs[\text{row}] \cdot \text{dp4a}(q4, xq) - m_1[\text{col}] \cdot xsum[\text{row}].$$

Activations are quantized to q8_1 once per prefill. It is gated `vulkan_q4k_dp4a2d_matches_default` (rel $< 1.5\times 10^{-2}$; the q8_1 activation quant adds $\sim 0.5\text{--}1\%$ on top of f32 reorder, a decode bug would be $\gg$ that). The default-flip ships it for dense ($w_{\text{off}}=0$, $m>1$) where the device advertises integer dot-product (queried via `PhysicalDeviceShaderIntegerDotProductFeatures`); otherwise the universal f32 2D tile. A gfx1151 engine now gets ~ 224 T/s Vulkan prefill by default, up from ~ 24 .

The practical ceiling, pinned by a tile sweep. At 5056 GFLOP/s the **dp4a-2D** kernel is $\sim 34\%$ of the ~ 14.8 TFLOP f32 peak. A tile sweep pins the bottleneck as *occupancy*, not VRAM/LDS traffic: register-blocking the inner **dp4a** is flat (the compiler already did it / added VGPRs that cost occupancy); BM=128 (half the weight re-read) is $3\times$ *slower* (register spill / occupancy collapse); BM=32 is slightly slower (re-read + barrier overhead). 64×64 is the sweet spot. The residual gap to f32-peak is the *inherent* Q4_K cost (per-sub-block scale application + in-kernel decode + the q8_1 activation pass), not a tiling defect — closing it needs a structurally different kernel (decode-amortized coopmat), not more tuning. This is the honest statement of where blind kernel-tuning ends on this GPU.

7 Measured dead-ends: the boundary map

The negatives are load-bearing: each is explained by the surfacing rule, and together they fence off the regimes. We report them because an inference team that does not know these boundaries will spend weeks re-discovering them.

Table 3: Things that were correct (bit-exact) but did not help, with the regime that explains why.

Change (bit-exact)	Result	Why (taxonomy)
Fold <code>quantize_q8_1</code> into the <code>dp4a</code> matvec (ROCm)	-22% (77.4 → 59)	Each matvec <i>block</i> re-quantizes the full activation into its own LDS (256× for a 2048-row proj) + a <code>__syncthreads</code> : redundant serialized work <i>injected</i> into a bandwidth-bound matvec. Under graphs the separate quantize launch was already free. <i>Violates the rule.</i>
Naive coopmat tensor-core prefill (Vulkan)	0.26× vs <code>dp4a</code>	Re-decodes the weight to f16 every K=16 step ($k/16$ passes); Q4_K decode swamps the fast <code>coopMatMulAdds</code> . Decode-bound, not tensor-core-bound.
1D weight-only LDS tiling (Vulkan)	0.21×	Weight re-read was not the bottleneck; few-heavy-workgroups lose to many-light register-tiled invocations. The lever was 2D (stage <i>both</i> operands), not 1D.
TILE.M=32 MoE GEMM (ROCm)	slower than 64	Less empty WMMA work but re-reads each expert weight in 2 tiles; the GEMM is weight-bandwidth-bound.
<code>dp4a</code> i-quant decode, model-level (GB10)	flat	Host/launch-bound decode: GPU finishes the faster matvec and stalls on host work. (Same kernel is 2.27× on ROCm-eager.)
<code>moe_route</code> fusion, model-level (CUDA)	flat	Repackages routing into one block-serial launch; CUDA decode is not router-launch-bound and CUDA has no expensive <code>gpu-sort</code> to eliminate (ROCm’s +13% came from that sort). <i>Repackaging, not eliminating.</i>
f32-native cast cut (ROCm)	flat	The casts are dtype-tangled around the F32-residual/F16-KV boundary; removing one shifts cost across the boundary.

Two of these are the *same kernel* as a kept win elsewhere (`dp4a` i-quant: ROCm 2.27× / GB10 flat; `moe_route`: ROCm +13% / CUDA flat). That is the cleanest possible confirmation of the taxonomy: the kernel did not change, the *binding resource* did. We keep both on the flat platforms anyway — bit-exact, decomplexing (one selector, fewer tables, matches llama), and latent for the regime where they bind — which is the right call precisely *because* the taxonomy says they cost nothing and may surface on the next model (e.g. a 256-expert router doubles the routing math).

8 The cross-backend law and model-level dilution

Stacking the per-backend work yields the model-level result of paper 05, restated as a law:

- **Decode is bandwidth-roofline parity** on every backend (0.98–1.04 $\times$): once the matvec streams weights at the memory ceiling, all engines converge, because decode throughput *is* memory bandwidth / model bytes. hanzo’s per-format decode matvec is at 88–99% of the measured roofline (rocprofv3 FETCH.SIZE), so there is nothing left to win there — and indeed every decode *kernel* win above surfaced only through *launch/work elimination*, never through faster matvec arithmetic.
- **Prefill is the frontier** (0.76–0.91 $\times$): it is compute-bound, so it rewards the matrix/tensor-core GEMM scheduling that llama.cpp has most heavily tuned. Every prefill lever in this paper (MoE qmmq 2.58 $\times$, dynamic M-tile, Vulkan dp4a-2D 9.35 $\times$) closes a chunk of this gap; the residual is real engineering (decode-amortized coopmat on Vulkan, the last $\sim 10\%$ of GEMM efficiency on CUDA), not a rounding error.

Model-level dilution. A kernel win of factor κ surfaces as $\approx 1 + f(1 - 1/\kappa)$ end-to-end, where f is the fraction of *binding-resource* time the kernel occupied. The Vulkan dp4a-2D 9.35 $\times$ kernel lifts engine prefill $\sim 24 \rightarrow 224$ T/s (f large, prefill is GEMM-dominated) but would be flat on decode ($f \approx 0$, decode is not the GEMM). The ROCm Q6_K 4.9 $\times$ kernel gives +17% ($f \approx 0.38$, its compute-bound fraction). The GB10 i-quant 1.71 $\times$ gives 0% ($f \approx 0$, host-bound). The law is just Amdahl’s law with the binding resource as the denominator — but the discipline it enforces is to *measure f before writing the kernel*.

9 A correctness boundary: the flash-attention repetition bifurcation

The sharpest result of this work is not a speed lever but a *correctness* one, and it maps the exact limit of fused attention on tensor cores. Qwen3-8B-Q4_K_M generates coherently under an eager attention (materialized $\text{softmax}(QK^\top/\sqrt{d})V$ in f32) but *collapses into token repetition* under flash-attention-2 on CUDA — byte-identical weights, same sampler, same box; only the attention kernel differs. The collapse is deterministic and appears at the first generated token.

The precision ladder, exhausted. Flash-attention-2 casts the f32 softmax probabilities $P = \text{softmax}(S)$ to the kernel element type before the $P \cdot V$ tensor-core matmul (`flash_fwd_kernel.h: convert_type<Element>(acc_s)`). We raised that precision in three proved-by-build steps, each verified on the running model at two seeds:

P representation	mantissa bits	exp	bare-trigger continuation
bf16	7	exp2f	"needle needle..."
f16	10	exp2f	"0123456789..."
split $P_{hi}+P_{lo}$	~ 20	exp2f	".matmat.matmat..."
split $P_{hi}+P_{lo}$	~ 20	expf (exact)	"1)2)3)4)5)..."
eager (reference)	23 (f32), single-pass	expf	"...Paris..." (coherent)

More precision only *moves the attractor token* (needle $\rightarrow$ digits $\rightarrow$ matmat); it never clears it. This is the signature of a knife-edge numerical bifurcation, not a localizable bug.

Everything else, eliminated. For the bare trigger (~ 7 tokens) the remaining flash-vs-eager differences are ruled out by structure, not guesswork: (i) the $QK^\top$ score MMA already accumulates in f32 (CUTLASS default), so S is f32-identical to eager; (ii) V is bf16 in the KV cache for *both*

paths, so the bf16-input/f32-accumulate MMA is numerically the eager matmul — there are no low bits of V to recover; (iii) with `kBlockN = 128` the trigger is a *single* K-tile, so the cross-tile online-softmax rescale executes zero times yet the collapse still appears; (iv) the `exp2f-vs-expf` gap is $\lesssim 2^{-23}$, orders below the 2^{-8} bf16- P error already shown non-curative — and this last one was *also* confirmed empirically: exact `expf` atop f32-class P still degenerates at both seeds (into a fresh "1)2)3)..." attractor), exhausting the precision-and-transcendental hypothesis space.

The boundary. What remains after all eliminations is a lone variable: P is ~ 20 -bit on the tensor-core path versus true 23-bit f32 in eager. Because tensor cores *require* $\leq \text{tf32/f16/bf16}$ inputs, a genuinely f32 $P \cdot V$ is the eager (CUDA-core) path — so on this hardware **fusion and f32-precision $P \cdot V$ are mutually exclusive**. The shipped fix routes bf16 attention to the eager kernel; it is not a stopgap but the *correct* answer, and it is essentially optimal: any stable variant must carry f32 $P \cdot V$, which forgoes tensor cores and thus lands at eager’s throughput regardless. Its only cost is long-context prefill ($2.66\times$ at 2409 tokens; short prompts and *all* decode are unaffected, since decode attention is a single query against the cache), and that cost is intrinsic to stability. The methodological point echoes the rest of this paper: the result was reachable only by *verifying against the running model* — five kernel edits that “compiled and committed” were disproved by the generated tokens before the boundary came into focus.

10 The characterized-but-open frontier: Vulkan decode

Paper 05’s parity claim is decode-on-AMD-via-ROCm. The Vulkan *decode* path is the one place hanzo is far from llama (8B Q4_K: 3.3 vs 41 T/s, $\sim 0.08\times$), and this program has now diagnosed it precisely rather than left it as “slow.” A `VK_PROFILE` breakdown of one decode token:

```
[VK_PROFILE] flush: dispatch=1336 barriers=903 record=0.766ms submit=1.406ms fence_wait=142.827ms
```

The diagnosis: it is **not** recording-bound (CPU record = 0.77 ms, so command-buffer/graph reuse would buy nothing) — it is **GPU-execution-bound on op-count serialization**. 1336 tiny $M=1$ dispatches chained through **903 global memory barriers** (`vk::MemoryBarrier`, full `SHADER.WRITE→READ`, on *true* data dependencies) serialize the GPU at $\sim 107 \mu\text{s/op}$ on compute that is only $\sim \mu\text{s}$. Per the taxonomy this is GPU-busy-bound decode, so the lever is *eliminating ops*, and the barriers are correct (real dependencies) — the only way to remove them is to fuse the dependent ops so the dependency lives *inside* a kernel.

10.1 Roadmap: the campaign ROCm already validated

This is not speculative: it is the exact campaign that took ROCm decode from launch/busy-bound to parity (§4), ported to Vulkan.

1. **Fuse the attention epilogue** — q/k RMSNorm + RoPE + KV-cache write into one kernel (ROCm’s `rope_norm_positions` twin), and residual-add + RMSNorm (`add_rmsnorm`). These *eliminate* per-op barriers, not just launches.
2. **Fuse the FFN** — gate+up shared-quantize, silu+mul.
3. **Fuse routing** — the `moe_route` kernel (already proven +13% on ROCm, where the eliminated sort is expensive — and Vulkan, like ROCm, pays for the sort).
4. **Uniform Vulkan GEMM for all 22 formats** — deletes the dequant fallback and the deadlock gate by construction (§6), the structural endpoint.

The honest status: **ROCm is at parity; Vulkan decode is the open frontier**, characterized down to the dispatch/barrier counts, with a roadmap whose every step has a measured ROCm precedent. The decompilation endpoint (axes §2) is a kernel DSL (CubeCL/Triton/MLIR-class) that writes the contraction once and lowers it to every backend, collapsing the backends $\times$ formats $\times$ ops product to backends+formats+ops — at which point a fusion is written once and surfaces on whichever backend’s regime it binds.

11 Verification methodology (so the numbers mean something)

Every claim in this paper rests on three controls, because inference benchmarks are unusually easy to corrupt:

- **Bit-exact gates.** A kernel ships only after `nbad=0` versus a CPU `to_float` oracle (documented f32-reorder / f16-rounding tolerances). Correctness is the precondition for a performance claim, not a follow-up.
- **Controlled A/B.** Decode A/Bs are device-map-pinned (`-n 0:48` on ROCm; all layers resident) and run in-binary via a fallback env toggle (`HANZO_*_FALLBACK`), so the *only* variable is the kernel. The headline “+28%” that an earlier pass reported was a measurement artifact — the auto-mapper offloading 1/3 of layers — and the controlled A/B showed the real number was +1.3%. We report the controlled number.
- **Kernel-isolated bench.** GEMM ladders use operands resident on-GPU and one final synchronize, because a per-iteration host upload/readback otherwise dominates and masks the kernel (the difference between the column kernel looking $1.2\times$ and $9\times$ slower).

Contamination caveat (stated, not hidden). The first evo Vulkan matrix in this pass was run under load 59 (a concurrent ROCm build + a model download + a browser) and is discarded; the numbers in §6–10 are from the idle box (load < 0.5). A benchmark on a contended box is not a slow benchmark, it is a wrong one.

12 What this paper does *not* claim

Single-stream only (no batched/serving throughput); the model-level numbers are paper 05’s single-model/single-quant pass, not re-swept here; Metal kernel-level work is summarized from its own audit, not re-derived; and the Vulkan-decode roadmap (§10.1) is characterized and preceded but *not yet shipped* — we label it frontier, not result. Every performance figure is a measured throughput or a kernel-isolated microbench on the named hardware; derived quantities (effective GB/s, M-fill %) are computed from measured values and labelled. No number here was invented or projected without the label “projection.”

13 Conclusion

The portable-engine thesis survives contact with the kernels: from one multi-backend codebase, hanzo-engine is at llama.cpp decode parity and within a bounded, well-understood prefill margin, and every step of that was a *measured* lever gated bit-exact. The transferable result is not any single kernel but the **surfacing rule and its taxonomy**: decode is bandwidth-roofline (win by moving fewer bytes or eliminating launches/work, never by faster arithmetic), prefill is compute-bound (win by tensor-core scheduling and tile fill), and the *same* kernel surfaces or vanishes depending on

which resource binds on a given (platform, regime). The dead-ends prove the rule as sharply as the wins — the -22% quantize-fold, the $0.26\times$ naive coopmat, the GB10 i-quant flat that is $2.27\times$ on ROCm. The one fully-characterized open frontier, Vulkan decode, is op-count-serialization-bound by profile, and its fix is the exact fusion campaign ROCm has already validated to parity. Optimize the binding resource; prove it bit-exact; measure f before you write the kernel.

A Reproducibility

Hardware. AMD Ryzen AI Max+ 395 “Strix Halo” (Radeon 8060S, gfx1151, RDNA3.5, 40 CU, ≈ 231 GB/s); NVIDIA GB10 (Grace-Blackwell, sm_121, CUDA 13, 121 GB unified, Linux aarch64); Apple M4 Max (137 GB unified, Metal). **Software.** hanzo-ml (the catalog spans the 0.11.x series) / hanzo-engine, built `--release` per box; `llama.cpp` stock HIP/CUDA/Metal/Vulkan builds as the baseline. **Models.** Qwen3 family GGUF (0.6B/4B/8B dense, 30B-A3B MoE) at Q8_0/Q4_K_M and the UD i-quants. **A/B protocol.** In-binary fallback env toggles (`HANZO.*FALLBACK`) so the kernel is the only variable; ROCm decode device-map-pinned `-n 0:NLAYERS`; GEMM ladders via the kernel-isolated `bench_matmul_q4k` / `bench_iquant_decode`; bit-exact gates via the named `*numeric` oracles in hanzo-cli. **Profilers.** `rocprofv3` (ROCm GPU-busy + `FETCH_SIZE` roofline), `nsys` (CUDA), `VK_PROFILE=1` (the in-engine per-batch dispatch/barrier/fence-wait breakdown). GPU-safety: ROCm/HIP processes are run strictly serially (concurrent kfd init on gfx1151 can wedge the driver), benches background-only.